

Compte Rendu TP 02

Test Infrastructure avec Netmiko

R4.ROM.09 — Outils DevOps

Léo Ponthieux

IUT de Roanne — R4.ROM.09 — 2025-2026

GitHub : <https://github.com/ShyeTF/R4.ROM.09>

1. Introduction

Ce TP s'inscrit dans la continuité du TP01. Les VMs routeur CSR1000v et Linux sont déjà configurées avec Ansible. L'objectif est d'écrire un script Python utilisant la librairie Netmiko pour valider automatiquement le bon fonctionnement de l'infrastructure.

Ce script sert d'outil de test de non-régression : on le relance à chaque modification de configuration Ansible pour s'assurer que tout fonctionne encore.



Emplacement des fichiers

Répertoire de travail : ~/TD_Ansible/test_python/

Fichier principal : test.py

GitHub : <https://github.com/ShyeTF/R4.ROM.09>

2. Tests du serveur Linux

2.1 Installation de Netmiko

Netmiko est une librairie Python qui simplifie les connexions SSH vers des équipements réseau (Linux, Cisco IOS, Juniper...). On l'installe avec pip3 :

```
pip3 install netmiko
```

Vérification de l'installation :

```
python3 -c "import netmiko; print(netmiko.__version__)"
```

```
4.1.2
```

On crée ensuite le répertoire dédié aux scripts Python :

```
cd ~/TD_Ansible
mkdir test_python
cd test_python
```

2.2 Test de connectivité sur le serveur web

On crée le fichier test.py. Le dictionnaire serv_web contient les paramètres de connexion SSH. Netmiko se connecte puis envoie la commande ping vers 8.8.8.8 pour tester l'accès internet depuis le serveur web.

```
serv_web = {
    'device_type' : 'linux',
    'ip'           : '192.168.3.2',
    'username'     : 'utilisateur',
    'password'     : 'toor',
    'port'         : 22,
    'verbose'      : False
}

connection = ConnectHandler(**serv_web)
test = connection.send_command('ping -c 4 -W 1 8.8.8.8')
```

L'option -W 1 dans la commande ping spécifie un timeout d'attente de réponse de 1 seconde par paquet. Sans cette option, si un paquet est perdu, ping attendrait indéfiniment la réponse avant de passer au suivant, ce qui bloquerait le script.

L'expression régulière capture les 4 groupes de chiffres dans la ligne de statistiques du ping :

```
# Ligne parsée : '4 packets transmitted, 4 received, 0% packet loss, time 3005ms'
capture = re.search(
    r'(\d+) \w+ \w+, (\d+) \w+, (\d+)% \w+ \w+, time (\d+)ms',
    test
)
if capture.group(1) == capture.group(2):
    print('+++ Test réalisé avec succès')
else:
    print('xxx Problème de connectivité')
    exit(-1)
```

Résultat obtenu :

```
>>> Serv_Web : Test de connectivité serveur Linux vers extérieur (dns google)
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=55 time=12.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=55 time=11.8 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=55 time=12.1 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=55 time=12.3 ms
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
+++ Test réalisé avec succès
```

2.3 Test résolution DNS sur le serveur web

On utilise nslookup avec l'option -timeout=1 pour tester la résolution DNS. Sans ce timeout, si le serveur DNS ne répond pas, nslookup attend très longtemps (30 secondes par défaut) avant d'abandonner, ce qui bloquerait le script.

```
test_dns = connection.send_command('nslookup -timeout=1 www.google.fr')

# La ligne 'Server:' contient une tabulation (\t) avant l'IP
# Exemple : 'Server:\t8.8.8.8' (\s+ matche tab et espaces)
capture_dns = re.search(r'Server:\s+(\d+\.\d+\.\d+\.\d+)', test_dns)

if capture_dns:
    print(f'+++ DNS OK - Serveur : {capture_dns.group(1)}')
else:
    print('xxx Pas de résolution DNS')
    exit(-2)
```

Résultat obtenu :

```
>>> Serv_Web : Test résolution DNS (nslookup www.google.fr)
```

```

Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   www.google.fr
Address: 142.250.75.195
+++ Résolution DNS OK — Serveur DNS utilisé : 8.8.8.8

```

Élément	Valeur / Résultat
Code retour	-2 si DNS KO, continue si DNS OK
ATTENTION tabulation	La ligne 'Server:' contient un \t. \s+ dans la regex matche tab ET espaces
Pourquoi -timeout=1	Sans timeout, nslookup bloque 30s si le DNS ne répond pas

2.4 Test du serveur web Apache (en local)

On utilise wget pour vérifier qu'Apache répond en local sur 127.0.0.1. Les options --timeout 1 -t 1 évitent que wget attende trop longtemps. On vérifie que le contenu de notre page HTML (nom de l'étudiant, mention R4.ROM.09) est présent dans la réponse.

```

# -q : silencieux, -O - : affiche le HTML dans stdout au lieu de sauvegarder
test_web = connection.send_command('wget --timeout 1 -t 1 -q -O - 127.0.0.1')

if re.search(r'index\.html|Ponthieux|R4\.ROM\.09|Apache', test_web, re.IGNORECASE):
    print('+++ Serveur Apache répond et sert le contenu attendu')
else:
    print('xxx Apache ne répond pas ou la page est incorrecte')
    exit(-3)

```

Résultat obtenu :

```

>>> Serv_Web : Test Apache en local (wget 127.0.0.1)
<!DOCTYPE html>
<html lang='fr'>
<head><meta charset='UTF-8'><title>TP Ansible - R4.ROM.09</title></head>
<body>
  <h1>Leo Ponthieux</h1>
  <p>Serveur web deploye automatiquement avec Ansible</p>
  <p>R4.ROM.09 - IUT de Roanne</p>
</body></html>
+++ Serveur Apache répond et sert le contenu attendu

```

2.5 Routeur depuis machine Ansible

Depuis la VM Ansible (192.168.4.2), on vérifie qu'elle peut joindre le routeur. Ce test est placé APRÈS les tests du serveur web car si le routeur est injoignable, c'est tout l'infrastructure qui est KO — on le détecte dès le début (section 3) avant même de tester les serveurs.

```

conn_ansible = ConnectHandler(**serv_ansible)
test_ansible_routeur = conn_ansible.send_command('ping -c 4 -W 1 192.168.1.17')
cap = re.search(r'(\d+) \w+ \w+, (\d+) \w+', test_ansible_routeur)
if cap and cap.group(1) == cap.group(2):
    print('+++ Machine Ansible joint le routeur')

```

```

>>> Ansible : ping vers routeur (192.168.1.17)
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
+++ Machine Ansible joint le routeur

```

2.6 Serveur web depuis machine Ansible

On complète le test en vérifiant que la VM Ansible joint également le serveur web. Ce test est placé APRÈS le ping routeur depuis Ansible, car si Ansible ne joint pas le routeur, il ne peut pas non plus joindre le réseau DMZ où se trouve le serveur web.

```
test_ansible_web = conn_ansible.send_command('ping -c 4 -W 1 192.168.3.2')
cap2 = re.search(r'(\d+) \w+ \w+', (\d+) \w+', test_ansible_web)
if cap2 and cap2.group(1) == cap2.group(2):
    print('+++ Machine Ansible joint le serveur web')
```

```
>>> Ansible : ping vers serveur web (192.168.3.2)
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
+++ Machine Ansible joint le serveur web
```

3. Tests sur le routeur Cisco

ATTENTION ordre des tests : les tests routeur sont effectués EN PREMIER dans le script (avant les tests serveurs Linux). En effet, le routeur est le cœur de l'infrastructure : il gère le NAT, le routage inter-VLAN et l'accès internet. Si le routeur est KO, aucun des tests suivants n'a de sens. L'ordre logique est donc :

- 1. Tests routeur (interfaces, connectivité, NAT, DHCP)
- 2. Tests serveur web (ping internet, DNS, Apache)
- 3. Tests depuis machine Ansible (vers routeur, vers serveur web)

3.1 Vérification des interfaces

On se connecte au routeur en mode enable (cisco_ios) et on vérifie chaque interface GigabitEthernet de 1 à 4 via une boucle. On contrôle que l'état est 'up' et que l'IP attendue est bien configurée.

```
conn_routeur = ConnectHandler(**routeur)
conn_routeur.enable()

interfaces_attendues = {
    1: {'ip': None, 'desc': 'WAN (DHCP)'},
    2: {'ip': '192.168.2.1', 'desc': 'LAN_ENTREPRISE'},
    3: {'ip': '192.168.3.1', 'desc': 'LAN_DMZ'},
    4: {'ip': '192.168.4.1', 'desc': 'ADMIN'},
}

for num in range(1, 5):
    resultat = conn_routeur.send_command(f'show ip interface GigabitEthernet{num}')
    if re.search(r'GigabitEthernet\d+ is up', resultat):
        print(f'+++ GigabitEthernet{num} : UP')
    if interfaces_attendues[num]['ip']:
        ip = interfaces_attendues[num]['ip']
        if re.search(re.escape(ip), resultat):
            print(f'    IP {ip} : OK')
```

Résultat obtenu :

```
+++ GigabitEthernet1 (WAN DHCP) : UP
+++ GigabitEthernet2 (LAN_ENTREPRISE) : UP
    IP 192.168.2.1 : OK
+++ GigabitEthernet3 (LAN_DMZ) : UP
    IP 192.168.3.1 : OK
+++ GigabitEthernet4 (ADMIN) : UP
    IP 192.168.4.1 : OK
```

3.2 Vérification de la connectivité

On envoie des commandes ping IOS depuis le routeur vers le serveur web et vers internet (8.8.8.8).

```
# Routeur → Serveur web (192.168.3.2)
test = conn_routeur.send_command('ping 192.168.3.2 repeat 4')
if re.search(r'!!!!', test):
    print('+++ Routeur joint le serveur web')

# Routeur → Internet (8.8.8.8)
test2 = conn_routeur.send_command('ping 8.8.8.8 repeat 4')
if re.search(r'!!!!', test2):
    print('+++ Routeur joint internet')
```

En IOS, le ping réussi affiche des '!' (un par paquet reçu). Un '.' indique un timeout. La regex r'!!!!' vérifie que les 4 paquets ont bien été reçus.

```
>>> Routeur → Serveur Web (192.168.3.2)
Type escape sequence to abort.
Sending 4, 100-byte ICMP Echos to 192.168.3.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (4/4), round-trip min/avg/max = 1/1/2 ms
+++ Routeur joint le serveur web avec succès

>>> Routeur → Internet (8.8.8.8)
!!!!
Success rate is 100 percent (4/4), round-trip min/avg/max = 12/13/14 ms
+++ Routeur joint internet avec succès
```

3.3 Tests supplémentaires proposés

Trois tests additionnels ont été intégrés au script pour renforcer la validation de l'infrastructure :

Élément	Valeur / Résultat
show ip nat translations	Vérifie que des entrées NAT sont présentes → confirme que le PAT fonctionne
show ip dhcp binding	Vérifie que des baux DHCP ont été distribués sur le réseau 192.168.2.x
show ip route	Vérifie la présence d'une route par défaut (0.0.0.0) → accès internet possible

```
# Test NAT
nat = conn_routeur.send_command('show ip nat translations')
if nat.strip(): print('+++ Table NAT contient des entrées : PAT actif')

# Test DHCP
dhcp = conn_routeur.send_command('show ip dhcp binding')
if re.search(r'192\.168\.2\.', dhcp): print('+++ Baux DHCP distribués')

# Test route par défaut
route = conn_routeur.send_command('show ip route')
if re.search(r'0\.0\.0\.0', route): print('+++ Route par défaut présente')
```

Conclusion

Le script test.py réalise une validation complète de l'infrastructure réseau automatiquement. L'ordre d'exécution des tests suit la logique réseau :

- Tests routeur EN PREMIER (interfaces, connectivité, NAT) → le routeur est la brique centrale
 - Tests serveur web (ping internet, DNS, Apache en local)
 - Tests depuis VM Ansible (joindre routeur et serveur web)
-